UNIT 1.PYTHON BASICS

1. Data Types in Python

Python is **dynamically typed**, meaning you don't need to declare the type of a variable.

```
x = 10   # automatically int
```

◆ Built-in Data Types

1. Numeric Types

- `int` → whole numbers
- `float` → decimal numbers
- `complex` → numbers with real and imaginary parts

```
a = 5
b = 3.2
c = 2 + 3j
```

---

2. Sequence Types

a) String (`str`)

- Immutable (cannot be changed)

```
s = "Hello"
```

b) List (`list`)

- Ordered, mutable (can change)

```
my_list = [1, 2, 3, "hi"]
my_list[0] = 10
```

c) Tuple (`tuple`)

- Ordered, immutable

```
t = (1, 2, 3)
```

---

3. Set Type

- Unordered, no duplicate values

```
s = {1, 2, 3, 3}  # Output: {1, 2, 3}
```

---

4. Dictionary (`dict`)

- Key-value pairs

```
student = {"name": "John", "age": 20}
print(student["name"])
```

---

5. Boolean (`bool`)

```
is_valid = True
```

□ 2. Input to Python Program

Python uses `input()` to take user input.

```
name = input("Enter name: ")
```

☞ Important:

- Input is always taken as **string**
- Convert when needed

```
num = int(input("Enter number: "))
```

Multiple Inputs

```
a, b = map(int, input().split())
```

🔤 3. String Basics (Detailed)

Strings are sequences of characters enclosed in quotes.

```
text = "Python"
```

◆ Operations

Indexing

```
print(text[0])   # P
```

Negative Indexing

```
print(text[-1])  # n
```

Slicing

```
print(text[0:4])  # Pyth
```

String Methods

```
text.lower()
text.upper()
text.strip()
text.replace("Py", "My")
```

Looping through String

```
for ch in text:
    print(ch)
```

➕ 4. Operators in Python (Detailed)

◆ Arithmetic Operators

```
+   # addition
-   # subtraction
*   # multiplication
/   # division
%   # modulus
//  # floor division
**  # exponent
```

◆ Comparison Operators

```
==  !=  >  <  >=  <=
```

◆ Logical Operators

```
and  or  not
```

◆ Bitwise Operators

```
&  |  ^  ~  <<  >>
```

◆ Assignment Operators

```
=  +=  -=  *=  /=
```

◆ Identity Operators

```
is  is not
```

◆ Membership Operators

```
in  not in
```

🔧 5. Basics of Functions

A function is a reusable block of code.

```
def function_name():
    # code
```

Example:

```
def greet():
    print("Hello")

greet()
```

◆ Function with Parameters

```
def add(a, b):
    return a + b
```

◆ Return Statement

- Returns value to caller

```
result = add(2, 3)
```

🔄 6. Passing Variables in Function Call

Variables are passed as arguments.

```
def show(x):
    print(x)
```

```
a = 10
show(a)
```

☞ Python uses **object reference passing**

- Mutable objects can change inside function
- Immutable cannot

🎯 7. Function Arguments (Types)

1. Positional Arguments

```
def sub(a, b):
    print(a - b)

sub(5, 2)
```

---

2. Keyword Arguments

```
sub(b=2, a=5)
```

---

3. Default Arguments

```
def greet(name="Guest"):
    print(name)
```

---

4. Variable-Length Arguments

*args (non-keyword)

```
def total(*nums):
    print(sum(nums))
```

**kwargs (keyword arguments)

```
def info(**data):
    print(data)
```

🎁 8. Modules in Python

A module is a file containing Python code.

◆ Importing Modules

```
import math
print(math.sqrt(16))
```

◆ Import Specific Items

```
from math import sqrt
```

◆ Create Your Own Module

File: `mymodule.py`

```
def greet():
    print("Hello from module")
```

Main file:

```
import mymodule
mymodule.greet()
```

🔄 9. Recursion (Detailed)

Recursion = function calling itself.

◆ Structure:

1. Base case (stopping condition)
2. Recursive case

Example: Factorial

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)
```

Example: Fibonacci

```
def fib(n):
    if n <= 1:
```

```
        return n
    return fib(n-1) + fib(n-2)
```

◆ Advantages

- Cleaner code for complex problems

◆ Disadvantages

- Uses more memory (stack)
- Can be slower than loops

✅ Final Summary

You now understand:

- Python data types and structures
- Taking input from users
- String operations and methods
- All major operators
- Functions and argument types
- Modules and code reuse
- Recursion and its working